

Mechatronic Laboratory (Robot: Advanced)

Team 12:

A.Abdullah Al

B.Catherine

TAN Daphne

Z.Md. Abdullah

G.C.Mohnish

1.Introduction

Our design is based on a four wheeled mobile robot with a unique claw or spiked wheel design combined with a raised chassis for high ground clearance. Taking advantage of the high stability, easy control, strong traction and high payload capacity of the 4-wheel design, we aim to overcome most of the obstacles and challenges in this robot challenge. Our main motivation was to design and fabricate a robot from scratch, as well as learn all the skills, knowledge, and understanding necessary to realize this goal. The originality of our robot lies in its spiked or claw-inspired wheel and its chassis design.

The spiked wheel, although at first looks orthodox, works well in overcoming most of the obstacles such as the uneven terrain, ditch, double roller and stairs. Similarly, our unique chassis design includes the brackets that hold our wheels as well as the body that holds the electronic circuit and hardware of our robot. At first glance the design may look straightforward but it has gone through multiple revisions to give us the most optimal design that enables us to overcome the remaining obstacles without much hassle.

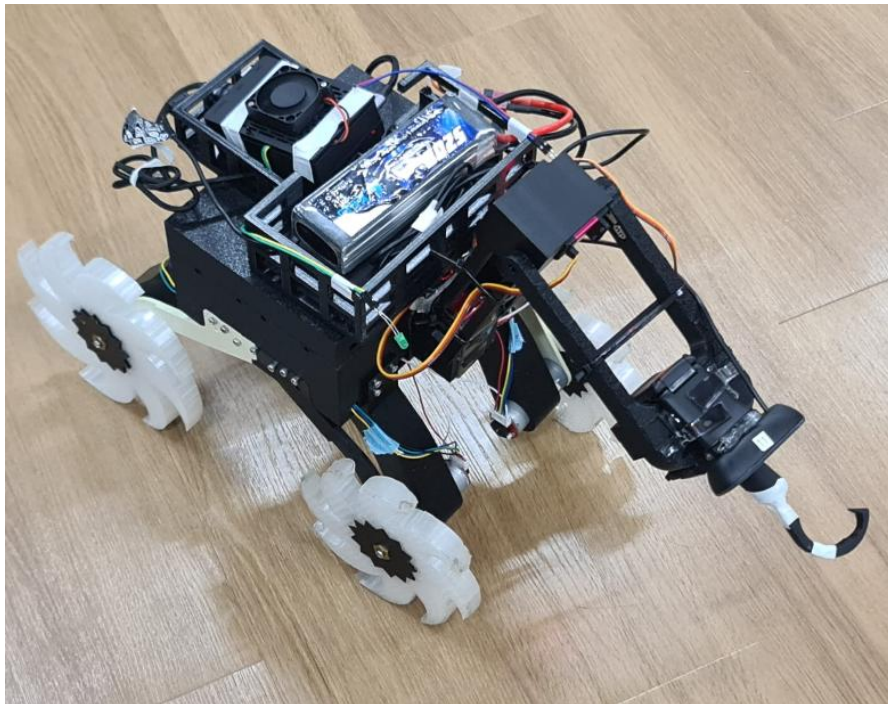


Figure I. Final Prototype of ELIBot

2. System Overview

2.1 Hardware Design: -

ELIBot's main body consists of three main components: the brackets that hold the brushed DC motors, the chassis that houses all the electronics including the microcontroller, motor drivers, and perfboards, as well as the cage that protects important components such as the Raspberry Pi 4 and the power bank.

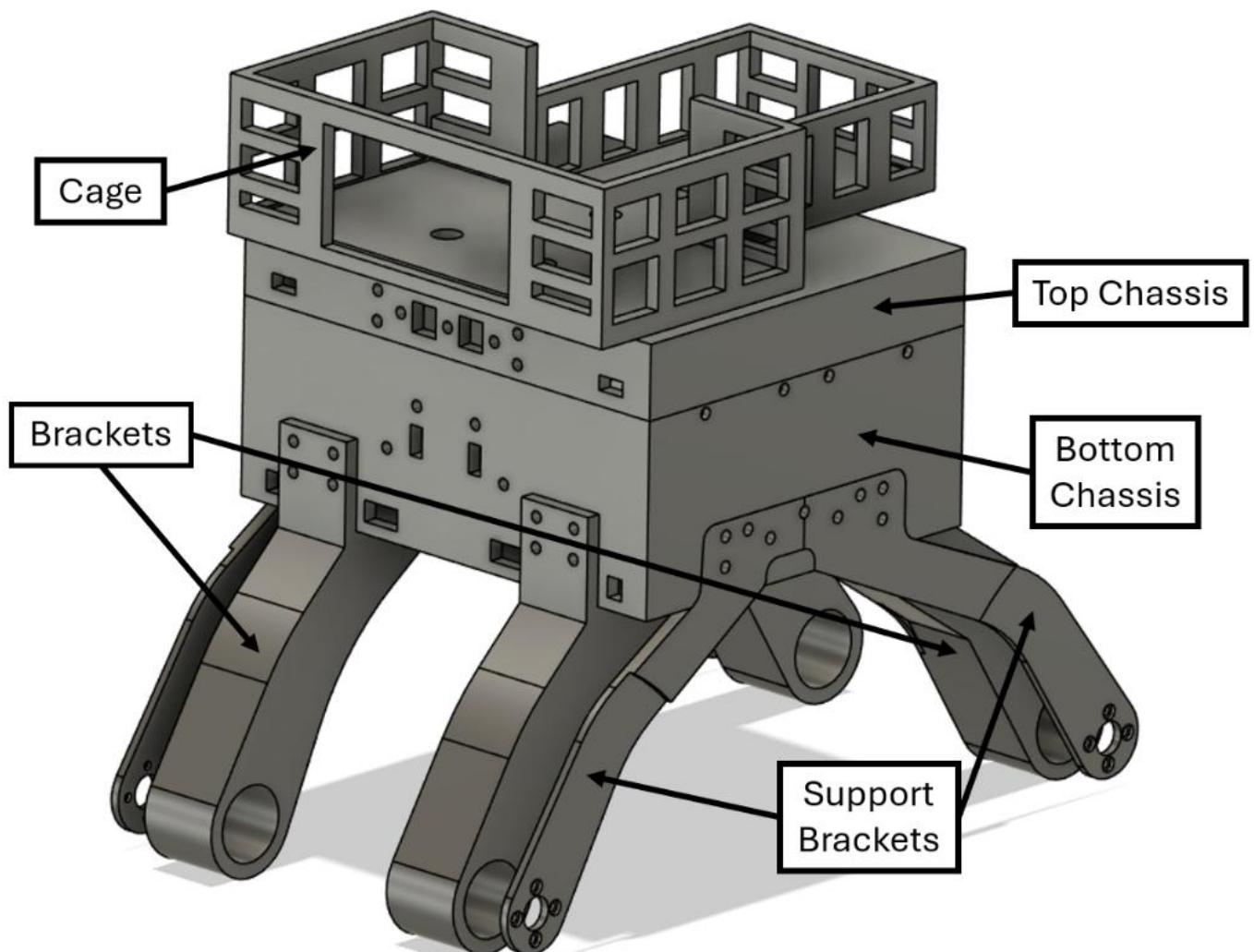


Figure II. Components of ELIBot Structural Design

2.1.1 Brackets: -

There are two types of brackets: main brackets and support brackets. The front main brackets are slightly shorter than the rear brackets, which serve multiple purposes such as helping in maintaining a consistent overall height, reducing the robot's overall footprint for a smaller turning radius, and compensating for the smaller clawed wheel used at the front compared to the back. Similarly, the front support brackets are shorter than the rear ones, as means of ensuring that both motors remain aligned, whilst also shifting the center of mass when climbing higher gradient surfaces such as the stairs and ditch, so that the robot did not tip over backwards due to weight distribution. The main purpose of both brackets however, is to provide stability and to hold the brushed DC motors in place when the clawed wheels make contact with the ground. All brackets are mounted to the chassis using M3 bolts and nuts, with each motor placed in the main bracket and secured by the support bracket. The support brackets further incorporate screw holes at they're ends, enabling each motor to be screwed in place to the bracket, prevents unwanted twisting of the motors and allowing all rotational torque to be used for turning the wheels.

2.1.2 Chassis: -

The chassis is a simple box design with a removable lid on top for easy access to the circuit and all internal components, such as the microcontroller, motor drivers, buck converters, and servo motor driver, allowing for easy adjustment or alteration. The chassis was built to be sturdy, not only to handle falls or failed attempts at clearing obstacles during testing, but also to protect all the internal components and circuitry during testing and throughout the robot challenge. Multiple holes were also included in the bodies design to allow for easy wired connections between the external motors and internal circuitry.

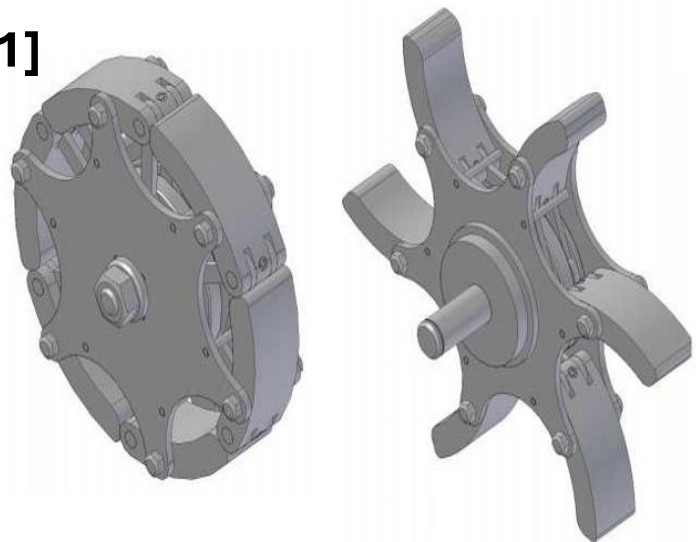
2.1.3 Cage: -

The main purpose of the cage is to protect and hold three main components, i.e. a 12V LiPo battery, a power bank, and the Raspberry Pi 4 Model B as well as their various connections, ensuring that the wires don't get loose or sustain any unforeseen damage during testing. Apart from protection, the cage also helps keep all the components in place and the wires organized for better management, so that they don't get in the way of the robot's locomotion. A lightweight design was chosen to reduce any unnecessary additional weight, which could have impacted the robot's function.

2.2 Clawed Design

The clawed design was inspired by the configurable wheel design [1][2], which has two forms: a closed configuration and an open configuration. With the help of gears, the wheel can dynamically switch between the two configurations

[1]



Although we initially considered going with the configurable design, we believed it was unnecessary for our wheel to switch between two states, and that a semi-open form would be sufficient to overcome the course and its obstacles. The main drawback of this decision was that it would cause jittery locomotion; however, the benefit was that it allowed us to follow a simpler design and fabrication process, without needing a complex gear mechanism or extra motors to not only rotate the wheel but also open and close it. Because of this quicker fabrication process, we were able to test a variety of wheels of different sizes, thicknesses, and materials until we found one that allowed us to complete the track.

The clawed wheel consists of two parts: an outer part and an inner part. The outer part is fabricated out of TPU95A with 8 claws, forming a circular wheel-like shape, whereas the inner part, which is attached to the DC motor via a shaft, allows the outer wheel to be attached, switched, or replaced easily without removing the shaft. A thread based-brass connector was used to attach to the shaft with sealant used to prevent the vibrations of the motor from loosening the threads, therefore preventing the wheel from falling off during testing and the robot challenge. The inner part of the wheel was designed to tightly fit the brass connector, and prevent it from moving. Due to the design of the connectors, specifically placed holes/gaps were incorporated into the inner wheels design so that the threads could be tightened from the outside using a simple L-shaped Allen key.

The wheel was also designed this way because a wheel made entirely out of TPU would be prone to deformation across the whole structure, due to the flexible nature of TPU. By instead fabricating the core, or inner part, out of PLA, the claws are able to deform on impact or when overcoming an obstacle such as uneven terrain or stairs, while the wheel as a whole remains stable, thanks to the rigid plastic core.

TPU was chosen due to its non-rigid nature, granting the outer wheel a degree of flexibility that otherwise would not be possible with other materials. Building the outer-wheel with this allowed for more adaptability to the course, as the TPU was capable of acting as a mild shock absorber when tackling obstacles such as the ledge and going down the stairs, whilst the materials properties, as well as gaps in the wheels claws helps maintained stability (such as when crossing the uneven terrain).

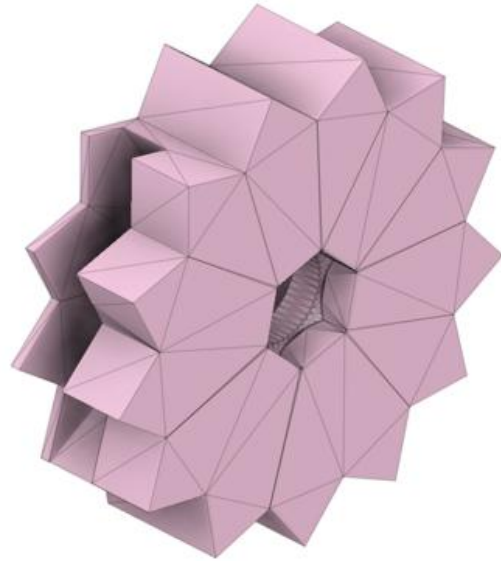
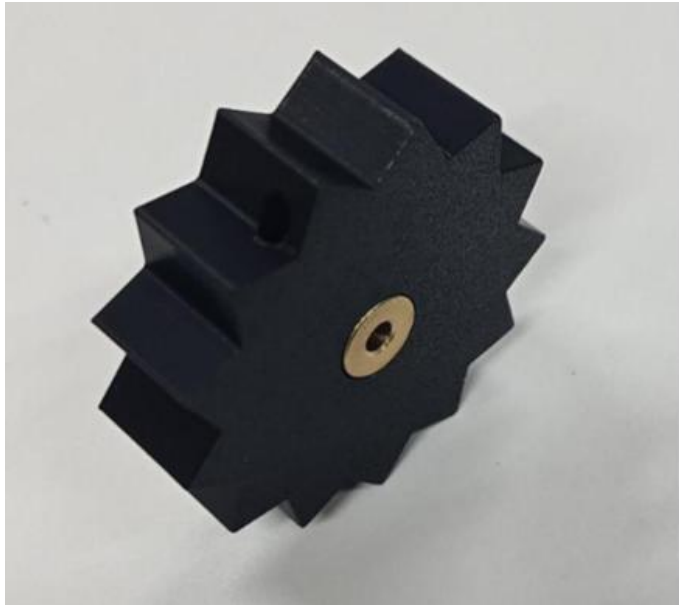


Figure III. Inner part of the wheel made from PLA

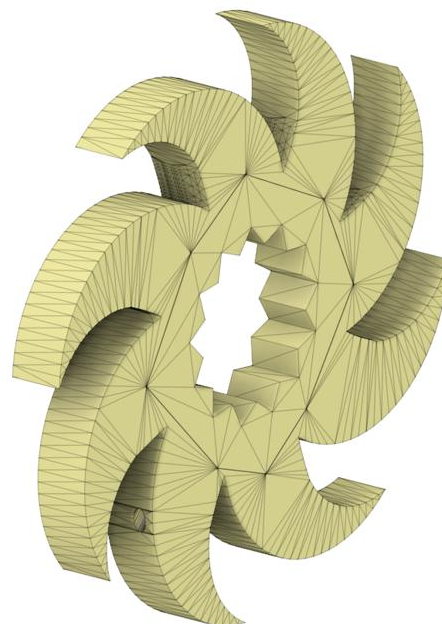
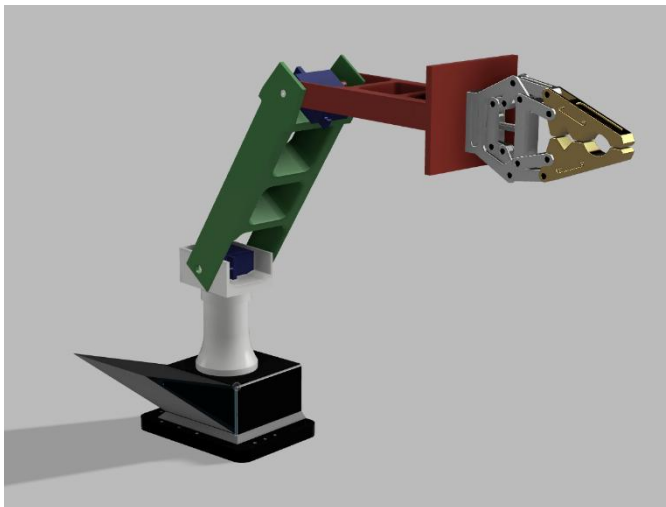


Figure IV. Outer wheel made from TPU

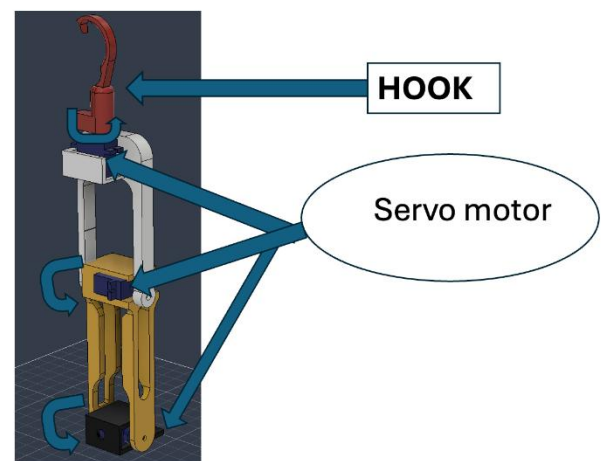
2.3 Manipulator Arm: -

The manipulator arm underwent several design iterations throughout the development process. The initial design was mounted on the top of the robot and featured three degrees of freedom (3-DOF). As the robot design evolved, the manipulator was redesigned to be lighter, more durable, and mechanically simpler. It was relocated to the front of the robot, where it functioned as an *elephant trunk*-like mechanism. The design was simplified to a single degree of freedom (1-DOF), reducing the robot's overall weight while improving front weight distribution. This enhanced the robot's stability, particularly when climbing and descending stairs.

The end effector also evolved during the design process. The original gripper design was unable to withstand the forces experienced during testing, especially when the robot tipped over. As a result, it was replaced with a curved hook that was stronger and capable of supporting the robot's weight without damage. The final manipulator assembly consisted of three motors: one at the base for arm rotation, one driving the second link, and one rotating the hook for object retrieval. This final configuration provided a lightweight, durable, and reliable solution for the robot's object manipulation tasks.



Initial design



Final design

Figure V. Manipulator Arm

2.4 Circuit Design: -

ELIBot is a 4-wheeled mobile robot equipped with a 2-degree-of-freedom manipulator arm, enabling it to both navigate its environment and manipulate objects within it. This dual functionality means ELIBot's electronics must handle two distinct actuation demands simultaneously: driving the four wheels for locomotion and articulating the arm for manipulation tasks.

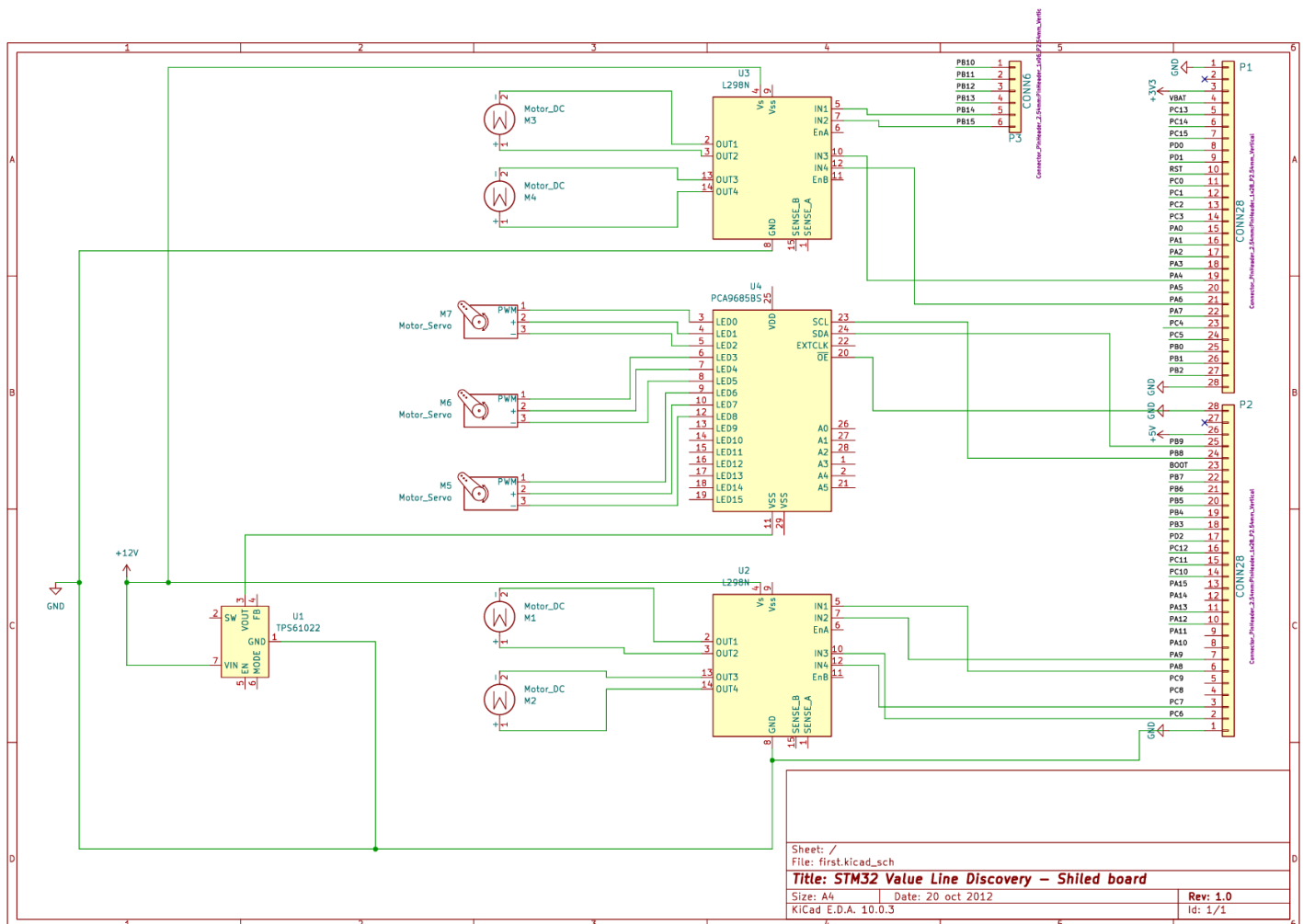


Figure VI. Overall Circuit Design used in ELIBot

As a result, ELIBot requires a circuit capable of not only powering and controlling four brushed DC motors, but also three servo motors, each with different control requirements and current draws. The four brushed DC motors handle the robot's mobility, driving the wheels via skid steering, while the three servo motors provide the joints of the manipulator arm, allowing it to reach, pivot, and grip.

To meet these requirements, the above circuit consists of four brushed DC motors, three servo motors, two L298N motor drivers, one PCA9685 16-channel servo driver, a buck converter, an STM32 NUCLEO-F091RC microcontroller, and a Raspberry Pi 4 Model B for controlling the different nodes. The two L298N drivers handle the higher current and bidirectional control needed for the DC motors, while the PCA9685 offloads precise PWM signal generation for the three servos, freeing up processing resources on the microcontroller. The buck converter steps down the main battery voltage to a stable, regulated level suitable for the logic circuitry, protecting sensitive components from voltage fluctuations. Together, the STM32 NUCLEO-F091RC and Raspberry Pi 4 form a two-tier control architecture, with the STM32 handling low-level, time-critical motor control and the Raspberry Pi managing higher-level decision-making and coordination between subsystems.

2.5 Software and Control

The software architecture of ELIBot is designed using a hierarchical control structure that separates high-level decision making from low-level actuator control, as illustrated in Figure VII.

High-level software is executed on the Raspberry Pi 4 Model B using the Robot Operating System 2 (ROS2), while the STM32 NUCLEO-F091RC microcontroller performs real-time control of the robot actuators.

Communication between the two processors is established through a USB serial interface operating at 115200 baud, enabling the Raspberry Pi to transmit control commands while the STM32 executes deterministic motor and servo control. This distributed architecture reduces the computational burden on the microcontroller while allowing computationally intensive perception algorithms to execute on the Raspberry Pi.

2.5.1 Robot Motion Control

The robot motion control system adopts a two-layer architecture in which the Raspberry Pi functions as the high-level controller and the STM32 serves as the real-time actuator controller. A dedicated ROS2 teleoperation node running on the Raspberry Pi receives keyboard input from the operator and converts it into compact serial commands that are transmitted to the STM32.

Upon receiving a command, the STM32 interprets the requested motion and independently controls the direction and speed of the robot actuators. The four DC drive motors are driven using pulse-width modulation (PWM) through two L298N motor drivers, implementing skid-steering locomotion by independently controlling the left and right wheel pairs. Motion commands determine the direction of rotation, while selectable PWM duty-cycle levels regulate the travelling speed of the robot.

The manipulator arm is actuated by three servo motors controlled through a PCA9685 16-channel PWM driver connected via the I²C interface. Rather than commanding abrupt position changes, the controller updates the desired joint positions incrementally while enforcing predefined mechanical limits for each joint. A slew-rate limiter gradually moves each servo towards its target position at a controlled angular velocity, producing smoother manipulator motion and reducing unnecessary mechanical stress on the servos.

To improve operational safety, the STM32 continuously monitors the arrival of motion commands through a watchdog mechanism. If communication with the Raspberry Pi is interrupted or no valid control command is received within the specified timeout period, the controller immediately stops all drive motors. This prevents unintended robot movement caused by communication failures or unexpected termination of the teleoperation software.

2.5.2 Vision Processing Pipeline

Robot perception is implemented as a modular ROS2 vision pipeline consisting of independent camera acquisition, perception, and status indication nodes. This modular publisher-subscriber architecture allows each processing stage to operate independently while exchanging information through ROS2 topics, simplifying debugging and future software expansion.

The camera publisher node continuously captures RGB images from the onboard USB camera using OpenCV and publishes them to the **/camera/image_raw** topic together with camera information. To reduce perception latency, the camera interface is configured to retain only the most recent captured frame, preventing outdated images from accumulating when image processing temporarily requires more time than the camera frame interval.

The published image stream is processed by the vision status node, which simultaneously performs QR code recognition and traffic light detection. QR code detection is implemented using the Pyzbar library after converting each image to grayscale. During software development, additional preprocessing techniques, including contrast enhancement and image upscaling, were evaluated; however, experimental testing showed no improvement in detection performance compared with direct grayscale decoding while increasing computational cost. Consequently, the final implementation performs QR recognition directly on the grayscale image.

Traffic light recognition is implemented using a hybrid detection approach. A YOLOv8 object detection model first identifies the location of traffic lights within the captured image. The detected region is subsequently analysed in the HSV colour space to determine whether the illuminated signal is red, yellow, or green. By combining deep-learning-based object localization with conventional colour segmentation, the system achieves reliable traffic light

recognition while reducing the computational complexity associated with directly classifying the entire image.

To maintain real-time performance, image acquisition and image processing are executed independently. The image subscriber stores only the newest received frame, while a dedicated worker thread continuously processes the latest available image. When image processing temporarily becomes slower than the camera frame rate, intermediate frames are discarded instead of queued, ensuring that the perception system always reacts to the most recent scene rather than progressively increasing system latency.

The outputs of the QR code detector and traffic light detector are combined into a single vision status message that is published whenever either detection state changes. To improve robustness against transient misdetections, both detection results are independently debounced before publication, ensuring that only confirmed state changes are propagated to the remainder of the system. When a confirmed transition to a green traffic light is detected, the vision node additionally publishes a notification message to the designated receiver topic used during the competition. A separate LED indicator node subscribes to the vision status topic and controls two GPIO-connected LEDs to provide immediate visual feedback indicating whether a QR code has been successfully detected.

The modular software architecture enables perception, communication, and actuator control to operate independently while exchanging information through standardized ROS2 topics. This separation of responsibilities improves software maintainability, facilitates debugging, and allows additional autonomous functions to be integrated into the system with minimal modification to the existing control framework.

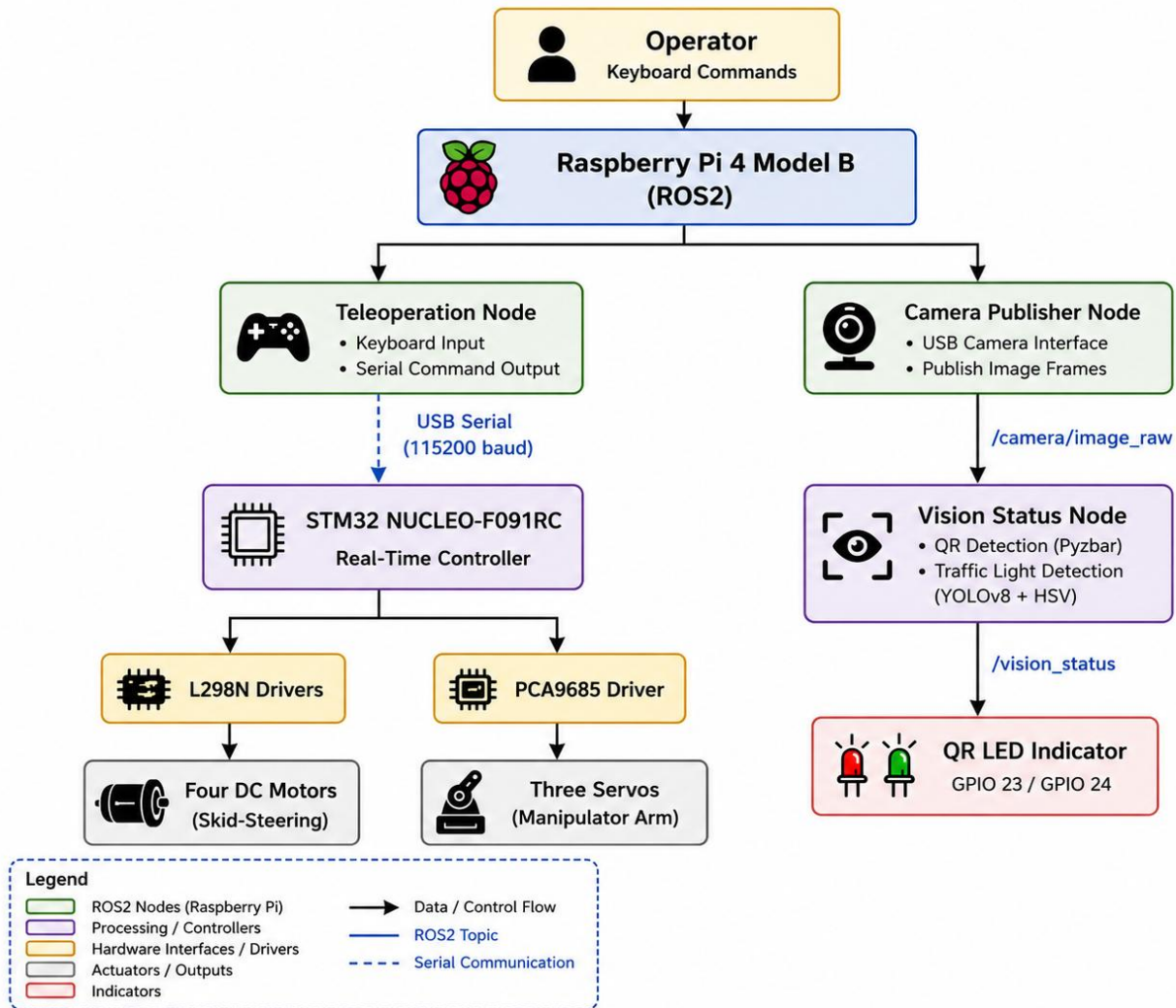


Figure VII. Overall software architecture of the ELIBot control and vision system.

3. Operating Instructions

After assembling the robot that is placing the motors and circuit in their designated positions and attaching both the inner and outer wheels in the correct orientation, we can proceed with the software setup.

First, we connect the microcontroller to the Raspberry Pi 4 and power the Raspberry Pi 4 using the power bank. Once the Raspberry Pi is powered, we connect it via HDMI to a computer and use OBS Studio to view the Raspberry Pi's environment on the computer screen. From there, we can open a terminal and begin running the required nodes.

Four nodes are required to control ELIBot:

1. The first node uses the on-robot camera to capture RGB frames and publishes this data to the topic */camera/image_raw*.
2. The second node subscribes to this topic to retrieve the RGB stream and detect the presence of a QR code, a traffic light, and the current light status displayed by the traffic light. This node only publishes a message when there is a change from the previous state, sending it to a second topic called */vision_status*.
3. The third node subscribes to the vision status topic and controls the external LED indicators according to the QR code detection result. The traffic-light notification message is published directly by the vision status node when a confirmed transition to a green signal is detected.
4. The final node captures keyboard input and forwards the corresponding control commands to the STM32 microcontroller, which performs the real-time control of the four DC motors and three servo motors while publishing actuator feedback to ROS2.

Once all nodes are running, we disconnect the laptop from the Raspberry Pi and connect the LiPo battery to check that all motor drivers are powered, and to confirm that we can control the robot by moving the wheels and manipulator arm.

4. References

- [1] R. Sell, G. Aryassov, A. Petritshenko, and M. Kaeeli, “Kinematics and dynamics of configurable wheel-leg,” in *Proc. 8th Int. DAAAM Baltic Conf. Industrial Engineering*
- [2] C. Zheng and K. Lee, “Wheeler: Wheel-leg reconfigurable mechanism with passive gears for mobile robot applications,” in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*,